LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Photo Print Order Application | Hypothesis Property-Based Testing

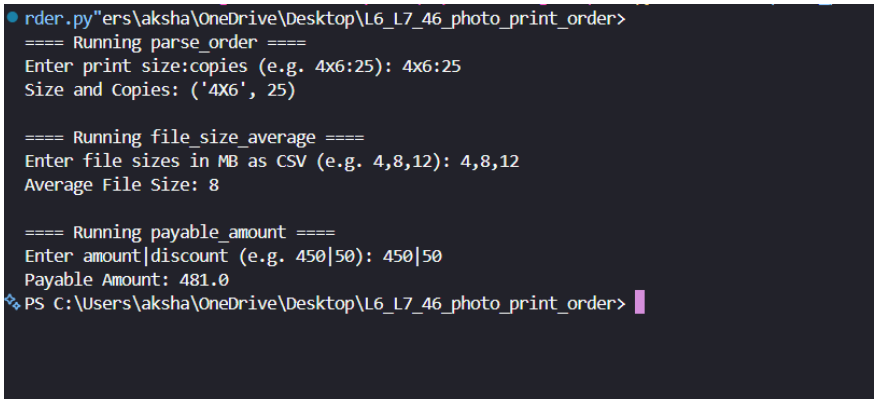
Objective

Use Hypothesis to generate unexpected and malformed inputs for the three application functions, observe failures in the original implementation, and improve input handling.

Application functions

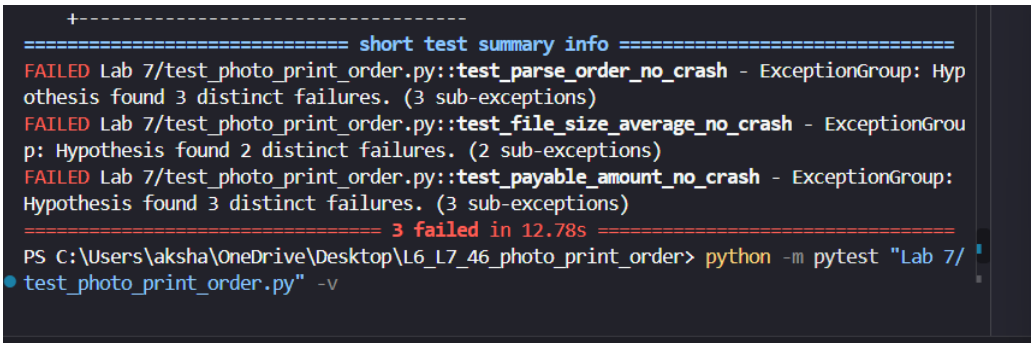
- **parse_order()** — parses print size and copy count.
- **file_size_average()** — calculates the average of comma-separated file sizes.
- **payable_amount()** — calculates the payable amount after tax and discount.

1. Original Application Output



The original application was run with the sample inputs 4x6:25, 4,8,12 and 450|50.

2. Hypothesis Failures Before Fixing



The initial fuzz run reported failures in all three no-crash tests. Hypothesis minimized inputs to examples including malformed separators, empty strings and None.

3. Fixed Application Output

```
PS C:\Users\aksha\OneDrive\Desktop\L6_L7_46_photo_print_order> python "Lab 7/photo_print_order.py"
==== Running parse_order ====
Enter print size:copies (e.g. 4x6:25): 4x6:25
Size and Copies: ('4X6', 25)

==== Running file_size_average ====
Enter file sizes in MB as CSV (e.g. 4,8,12): 4,8,12
Average File Size: 8

==== Running payable_amount ====
Enter amount|discount (e.g. 450|50): 450|50
Payable Amount: 481.0
PS C:\Users\aksha\OneDrive\Desktop\L6_L7_46_photo_print_order> \
```

After updating input validation, the sample inputs produced the expected results: ('4X6', 25), average file size 8, and payable amount 481.0.

4. Hypothesis Tests After Fixing

```
PS C:\Users\aksha\OneDrive\Desktop\L6_L7_46_photo_print_order> python -m pytest "Lab 7/test_photo_print_order.py" -v
============================= test session starts ==============================
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\aksha\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\aksha\OneDrive\Desktop\L6_L7_46_photo_print_order
plugins: anyio-4.13.0, deepeval-4.1.4, hypothesis-6.170.15, langsmith-0.9.7, asyncio-1.4.0, repeat-0.9.4, rerunfailures-16.4, xdist-3.8.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 3 items

Lab 7/test_photo_print_order.py::test_parse_order_no_crash PASSED                [ 33%]
Lab 7/test_photo_print_order.py::test_file_size_average_no_crash PASSED          [ 66%]
Lab 7/test_photo_print_order.py::test_payable_amount_no_crash PASSED             [100%]Running teardown with pytest sessionfinish...


============================== 3 passed in 2.08s ===============================
PS C:\Users\aksha\OneDrive\Desktop\L6_L7_46_photo_print_order>
```

All three Hypothesis tests passed after the application was hardened.

Conclusion

Fuzz testing exposed input-validation failures in each function. Defensive checks were added for malformed input, missing values, invalid numeric values, negative values and non-finite monetary values. The same three fuzz tests were rerun successfully.